DYNAMIC AND HYBRID CONDITIONING FOR COMPOSITIONAL IMAGE RETRIEVAL
Deep Learning Assignment 2026

Updated: 2026/05/21 — Version: 1.2

1 INTRODUCTION

This assignment investigates the problem of compositional image retrieval, a highly challenging computer vision task that entails retrieving target images based on a complex combination of multimodal conditions. You will build upon the foundational work on semantic compositionality introduced by Berasi et al. (2025) and the recent architectural advancements proposed in CLAY Lim et al. (2026).

Recent literature has made significant progress in latent space disentanglement. Specifically, GDE Berasi et al. (2025) introduced a framework designed to decompose a densely entangled embedding – representing multiple semantic concepts – into separate structures within the latent space. CLAY Lim et al. (2026) expands upon this paradigm by reframing the embedding space of pretrained Vision-Language Models (VLMs) as a text-conditional similarity space. This reframing permits highly efficient conditioned retrieval against fixed visual embeddings by fully decoupling the textual conditioning process from the visual feature extraction pipeline.

Despite these advancements, a critical limitation remains: when processing multiple semantic conditions, the CLAY pipeline relies on a naïve stacking or concatenation of embeddings prior to applying Singular Value Decomposition (pre-SVD). This approach is architecturally rigid and demonstrably suboptimal, as it offers no direct, dynamic control over how multiple, potentially conflicting conditions should be weighted, attended to, or combined.

Your primary objective is to develop, implement, and rigorously evaluate a dynamic similarity metric where the conditioning process intelligent integrates multiple conditions, treating them as either positive (additive) or negative (subtractive) constraints. Formally, given a reference image $v_{ref} \in \mathcal{V}$, a set of positive textual constraints $\mathcal{T}^+ = \{t_1^+, \ldots, t_n^+\}$, and a set of negative textual constraints $\mathcal{T}^- = \{t_1^-, \ldots, t_m^-\}$, your task is to design a fusion module Φ that yields a composite query embedding. This query embedding must then successfully retrieve target images v_{target} that preserve the core identity of v_{ref} while satisfying all specified additive and subtractive semantic modifications.

2 DATASET

The visual retrieval tasks in this assignment will utilize established attribute classification datasets. These datasets have been specifically selected because they map well to textual conditioning tasks, accommodate limited computational resources, and yet provide a highly challenging benchmark for fine-grained conditional retrieval.

You will evaluate your methods on the standard dataset **CELEB-A** Liu et al. (2015), a large-scale face attribute dataset with more than 200K images, each annotated with 40 distinct binary attributes.

Other datasets that might be useful thanks to their attribute annotations are:

- **OVAD** Bravo et al. (2023): an object-centric dataset that provides dense attribute annotations, ideal for testing fine-grained visual reasoning.
- **Animals With Attributes 2 (AwA2)** Xian et al. (2018): a dataset featuring 50 animal classes paired with 85 semantic attributes, commonly used for zero-shot learning and compositional evaluation.

You are not required to evaluate on these datasets, but they can be useful resources to develop your project.

During the assignment, you are expected to systematically evaluate your dynamic retrieval framework on the official test split of the CELEB-A dataset and provide a comprehensive, statistically sound interpretation of your achieved results.

3 TASK DESCRIPTION

The core goal of this project is to implement a flexible, expressive architecture that replaces the rigid pre-SVD embedding stack of existing literature with an intelligent representation fusion mechanism, enabling nuanced and controllable compositional retrieval.

You are expected to propose and develop a novel methodological approach. There is no strict restriction on the specific architectural paradigm, and you are entirely free to explore both *training-free* (*e.g.*, prompt engineering, zero-shot attention scaling, latent space arithmetic) and *training-based* strategies (*e.g.*, training lightweight adapter modules, cross-attention networks, or interpretability-based sparse autoencoders (SAEs) solutions). We strongly advise designing a *lightweight* model to facilitate rapid prototyping, faster iterations, and efficient experimentation.

Specifically, your proposed solution must address the following core challenges:

1. Expressive multimodal conditioning. Your architecture must natively accept and process multiple textual conditions alongside a visual reference. The model must learn or inherently understand how to dynamically weigh these inputs. For instance, given a reference image v_{ref} and the textual constraints `+glasses` and `−red hair`, the retrieval function must score images highest if they share the latent identity of the reference, explicitly contain glasses, and explicitly do not contain red hair (*i.e.*, any alternative hair color is valid). You must define how these positive and negative constraints interact in the embedding space.

2. Overcoming the naïve SVD bottleneck. The standard concatenation of textual and visual embeddings prior to SVD dramatically limits the expressiveness and relational reasoning of the model. You are expected to explore more advanced fusion mechanisms. Potential avenues include, but are not limited to, cross-attention layers, gating mechanisms, non-linear projection heads, or contrastive alignment strategies that dynamically re-weight features based on the provided text conditions.

3.1 EVALUATION PROTOCOL

To ensure a standardized evaluation protocol and facilitate fair comparisons across all submissions, we will provide a predefined set of evaluation queries and a corresponding ground-truth JSON file. This benchmark set will encompass multiple levels of complexity:

- **Simple queries:** modifications involving a single attribute (*e.g.*, `+glasses` or `−red hair`).
- **Composed queries:** simultaneous modifications of multiple semantic attributes (*e.g.*, `+glasses`, `−smile`, or `+sad`, `−blue eyes`).

The list of mandatory evaluation queries is provided in the "`celeba_evaluation.json`" file on Google Drive. The link is also available on Moodle. We report the list here, too, for your convenience. Note, however, that you should *always* refer to the JSON file.

We request that you evaluate your model on the following queries:

- `+ Smiling`
- `+ Eyeglasses`
- `- Heavy Makeup`
- `+ Male`
- `- Young`
- `+ Blond Hair`
- `+ Mustache`
- `+ Eyeglasses & - Smiling`

- `- Male & - Mustache`
- `+ Chubby & - Young`
- `- Smiling & + Eyeglasses & + Wearing Hat`
- `+ Wearing Lipsticks & - Heavy Makeup & + Smiling`

Beyond the mandatory benchmark, you are strongly encouraged to evaluate your architecture on additional, custom queries to broaden the scope of your analysis. If you choose to do so, please explicitly detail these custom queries in your final report and provide a rigorous interpretation of the resulting insights.

3.1.1 GROUND-TRUTH FORMULATION

Due to the inherent sparsity of the large attribute spaces, finding an exact match for all $N - 1$ non-queried attributes is often impossible. To robustly evaluate your models, the provided JSON establishes ground-truth targets based on a relaxed Hamming distance metric. For a given reference image v_{ref} and a query, a target image is considered a valid ground truth if and only if:

1. it strictly satisfies the positive / negative constraints of the query;
2. its remaining attributes have a maximum Hamming distance ≤ 2 from v_{ref}, ensuring the core visual identity is preserved.

To guarantee statistical significance during evaluation, the benchmark only includes source images that possess ≥ 5 valid ground-truth targets within the test set. You must restrict your evaluation strictly to the source image indices (keys) provided in the JSON for each respective query.

3.1.2 HOW TO USE THE GROUND-TRUTH

Let us refer to the provided JSON ground-truth file as `gt`. The file is structured as a list of dictionaries, where each element corresponds to a specific textual query (*e.g.*, `gt`[0] for `"+ Smiling"`, `gt`[1] for `"+ Eyeglasses"`).

Each dictionary in `gt` shares the exact same structure with two primary keys:

- `query`: The textual query string as reported in this document (*e.g.*, `"+ Smiling"`).
- `ground_truth`: A dictionary mapping source (query) image indices to a list of target image indices.

CRUCIAL WARNING: DATASET INDEXING VS. FILENAMES

The keys in the `ground_truth` dictionary represent the **integer indices of the PyTorch dataset object**, *not* the physical filenames on your disk.

For example, consider a case where the first key in `ground_truth` for `"+ Smiling"` is the string `"13"`. Assuming you have initialized your dataset as:

```
celeba = CelebA(root=data_root, split="test", download=False)
```

The wrong way (do NOT do this):
Do *not* use this index to manually construct a file path or load an image from the filesystem. If you attempt to load the image manually using the index string to guess the filename:

```
# THIS IS WRONG. It will load the incorrect image shown in Fig. 1(a)
Image.open(
Path(dataset_root) / celeba.base_folder / "img_align_celeba" / "000013.jpg"
)
```

The right way:
You must access the image directly via the dataset object using the integer conversion of the key:

(a) File `000013.jpg` (Incorrect).

(b) File `182651.jpg` (Correct).

Figure 1: Comparison between manual filename loading (left) and proper dataset indexing (right) for index `13`.

```
# THIS IS CORRECT. It will load the intended image shown in Fig. 1(b)
image, label = celeba[13]
```

Why does this happen?
The test split of CelebA is a subset of the full dataset. The PyTorch `CelebA` class maps internal sequential indices $(0, 1, 2, \dots)$ to the actual underlying filenames.

If you inspect the dataset metadata via `celeba.filename[13]`, it returns `"182651.jpg"`. This clearly demonstrates that dataset index `13` does *not* point to the physical file `"000013.jpg"`. Always use the dataset object (`celeba[idx]`) to fetch your samples.

3.1.3 Evaluation metrics

For each query, you are required to compute and report the following metrics at $K = 1$, $K = 5$, and $K = 10$, averaged across all valid source images:

- **Recall@K (hit rate):** the primary metric for this task. It is a binary indicator of whether your model successfully retrieved *at least one* valid ground-truth image in its top K predictions.

$$Recall@K = \begin{cases} 1, & \text{if } |\mathcal{R}_K \cap \mathcal{G}| > 0 \\ 0, & \text{otherwise} \end{cases} \tag{1}$$

  where $\mathcal{R}_K$ is the set of top K retrieved images and $\mathcal{G}$ is the set of ground-truth images.
- **Precision@K:** a secondary metric evaluating the density of correct matches in your top K results.

$$Precision@K = \frac{|\mathcal{R}_K \cap \mathcal{G}|}{K} \tag{2}$$

3.2 Project roadmap and baseline

To ensure structured progress, we recommend adhering to the following development pipeline:

1. **Data exploration and preprocessing:** familiarize yourself with the attribute annotations of the datasets. Write scripts to validate that viable target images (which satisfy various combinations of $+/-$ conditions) actually exist within the retrieval corpus for your generated queries.

2. **Offline feature extraction:** extract visual features for the entire dataset corpus using a pretrained VLM (*e.g.*, CLIP). Following the methodology in CLAY, this visual database should be constructed once offline and kept frozen, ensuring efficient retrieval during your experiments.
3. **Vanilla zero-shot baseline:** implement a simple baseline utilizing a standard, unmodified CLIP model. In this setup, perform naïve latent space arithmetic (*e.g.*, $v_{target} \approx v_{ref} + t_{glasses} - t_{red\_hair}$) without any learned fusion or SVD logic. This will serve as your lower-bound performance metric, helping you familiarize yourself with the evaluation pipeline and VLM behaviors.
4. **Method development:** implement your novel fusion mechanism and iteratively compare its performance against the zero-shot baseline.

3.3 Model to use

You should use **CLIP ViT-B/32** from HuggingFace.

You can find it at https://huggingface.co/openai/clip-vit-base-patch32. While you are welcome to test other models, this is the one we request everyone to use. If you evaluate other models, you should list results in your final report (notebook).

4 Evaluation and deliverables

Your final delivery must be self-contained within a single Jupyter Notebook hosted on Google Colab. The notebook must contain your complete codebase, logically divided into multiple executable cells in a clean, modularized, and highly readable format.

Furthermore, you are required to utilize the Markdown cells within the Notebook to provide a comprehensive project report. Your notebook should read like a detailed report interwoven with executable code.

In your report sections, you are strictly required to provide:

- **Methodological description:** a detailed, formal overview of the solution you developed. This must include a mathematical description of your architecture, the forward pass, and the loss functions governing the training process (if applicable). Clearly highlight your original contributions and adequately cite relevant literature.
- **Experimental setup:** a rigorous description of the training and evaluation strategy. Extensively motivate your methodological choices, including network capacity, optimizer selection, hyperparameter tuning, and data sampling strategies.
- **Results and discussion:** an extensive presentation of your findings. You must report standard retrieval metrics (Recall@K). Organize your scores in comparative tables, and include charts depicting learning curves (if applicable), qualitative retrieval examples (successes and failure cases), and any other visual representations that aid in understanding the model's behavior.

Your project will be evaluated holistically according to the following criteria:

- **Originality and creativity:** the novelty and elegance of the proposed fusion mechanism.
- **Methodological thoroughness:** the rigor of your experimental design and validation strategies.
- **Report clarity:** the quality, formality, and clarity of the written scientific exposition.
- **Empirical performance:** the retrieval accuracy (*e.g.*, R@1, R@5) achieved on the test sets compared to the baseline.
- **Code quality:** the readability, modularity, and efficiency of your Python implementation.

5 GROUP REGISTRATION

This project is intended to be developed by groups of 2 or 3 students. While individual submissions are permitted, collaborative work is highly encouraged.

Please formalize your team by registering via the provided Google Form. If you are currently unassigned but wish to join a group, please indicate this preference on the form. We will try to match you with other students seeking group members.

Link to the Google Form.

6 POLICIES

We strictly require that you do not base your project on an existing, fully-fledged GitHub repository developed by third parties. While your proposed architecture does not need to be exceedingly complex, you are expected to build it from scratch, or build directly upon the starter code provided during the laboratory sessions.

If absolutely necessary, small, specific utility snippets (*e.g.*, a specific PyTorch tensor operation) may be borrowed from open-source repositories, provided they are explicitly commented and cited in your code. This policy is enforced to ensure you gain practical, hands-on experience in engineering a deep learning pipeline, rather than merely assembling pre-existing black-box components.

You are welcome to discuss high-level concepts and theoretical approaches with members of other groups. However, sharing source code across groups is strictly forbidden. Please be advised that all final notebook submissions might be processed through automated plagiarism detection utilities. Any violation of these academic integrity policies will result in severe disciplinary action against all involved parties, including those who lent their code. It is the responsibility of every individual to strictly adhere to these guidelines.

REFERENCES

Davide Berasi, Matteo Farina, Massimiliano Mancini, Elisa Ricci, and Nicola Strisciuglio. Not only text: Exploring compositionality of visual representations in vision-language models. In *CVPR*, 2025.

María A. Bravo, Sudhanshu Mittal, Simon Ging, and Thomas Brox. Open-vocabulary attribute detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 7041–7050, June 2023.

Sohwi Lim, Lee Hyoseok, Jungjoon Park, and Tae-Hyun Oh. Clay: Conditional visual similarity modulation in vision-language embedding space. In *CVPR*, 2026.

Ziwei Liu, Ping Luo, Xiaogang Wang, and Xiaoou Tang. Deep learning face attributes in the wild. In *Proceedings of International Conference on Computer Vision (ICCV)*, December 2015.

Yongqin Xian, Christoph H Lampert, Bernt Schiele, and Zeynep Akata. Zero-shot learning—a comprehensive evaluation of the good, the bad and the ugly. *IEEE transactions on pattern analysis and machine intelligence*, 41(9):2251–2265, 2018.